17-214/514: Agentic Software Development

Concurrency and Asynchrony

Hammad Ahmad and Claire Le Goues • Monday, November 2, 2026 • Lecture

Administrivia

- Midterm 2 is on Wednesday. In this room, at 11am.
  - Same deal as last time. 80-minute exam, pen and paper.
  - You can bring 4 pages of US letter-size cheat sheet, front and back.
  - We posted some sample questions.
- A5 was released last Wednesday. Checkpoint due this Friday.
- Grades for A3 have been published on Gradescope.
- Hammad's office hours rescheduled to Mondays at 3:30pm. Same location.

What did we talk about last time?

- Anything one copy keeps in memory, the other copies can't see. State needs a home outside the copies.
- A performance claim is testable when it's a number, checked by a load test with traffic you designed.
- The same load generator can check correctness as well as speed. As a race test, it found double bookings at three copies.
- Stored data outlives code. Change its shape with expand, migrate, contract.
- Every copy of data (a cache, a read model) should have an owner and a decision about how fresh it has to be.
- Copies of data can give stale reads and lost updates.
- The data model is a part agents shouldn't change freely, but also one of the best specs to hand them.
- When an agent says the tests pass, ask where they ran.

Learning goals for today

- Walk through an interleaving of two requests and find where shared state can go wrong
- Recognize check-then-act races, lost updates, deadlock, and stale reads between threads
- Choose a remedy for a race (a lock, the database, immutability, confinement) and say what it costs
- Design a test that forces one interleaving and fails on every run while the race is there
- Spot async calls that aren't awaited and races across an await
- Explain why passing tests say little about concurrent code (generated or not)

Does one copy need a lock?

The agent’s plan from last week: "Wrap the check-then-insert in book() in a threading.Lock, so two requests can’t book the same slot at once".

The race test passed at one copy but failed at three.

Concurrency we don’t choose

Common sources of concurrency

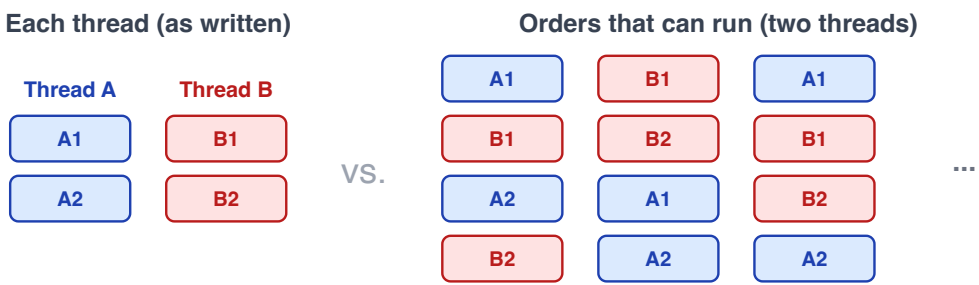
- **Users:** Two students click "book" within milliseconds of each other. The booking code doesn't start threads, but the server can run both requests at once.
  - Many web servers run each request on its own thread (e.g., Tomcat for Java, Puma for Ruby), so two `book()` calls can run at the same time.
- **Background work:** Scheduled jobs and queue workers run alongside requests (e.g., a nightly job that cancels expired bookings).
- **Cores:** A server can run requests in parallel on several cores.
  - Even one core switches between threads, so races don't need several cores.
- **Networks:** Messages from different machines can arrive in any order.

Threads, processes, and async tasks

- **Threads** each run their steps in order. Several threads share memory and can run at the same time (interleaved on one core, or in parallel on several).
  - Sharing is fast but easy to get wrong.
- **Processes** have separate memory and share data only on purpose (through a database or a message).
  - This is safer, but each one costs more to start and run.
- **Async tasks** (e.g., in JavaScript or Python's asyncio) take turns on one thread, and pause while they wait for something slow.
- In each case, the question is what happens when two of them touch the same data.

Reasoning about execution top to bottom

- In one thread, line 2 runs after line 1, so we can read code top to bottom.
- The operating system decides which thread runs when, and it can switch to another thread after any step.
- With two threads, their lines can mix in many orders, and the order can change from run to run. One such order is called an **interleaving**.
  - Two short methods running at once already allow many interleavings. Most people usually review just one of them.



The double-booking race

Reminder: ADR 007

```
# ADR 007: Prevent double-booking with an overlap check in the domain
...
Alternatives: A database exclusion constraint. Rejected because it ties
          the rule to one storage engine and hides it from the tests.
Consequences:
  + One clear, testable place for the rule.
  + No database-specific features required.
  - Two concurrent requests can still race (revisit under concurrency).
```

We recorded this race as a known cost when we made the decision. Let’s now look at the race and the alternative we rejected.

The check and the insert

```
Booking book(User u, Room r, TimeRange slot) {
  if (hasActiveOverlap(r, slot)) {          // check
    throw new SlotTaken();
  }
  return store.insert(new Booking(u, r, slot));  // act
}
```

- With one request at a time, this is correct, and every test passes.
- But it's two steps (a check, then an insert), with a gap between them.
- Another request can book the same slot inside that gap.

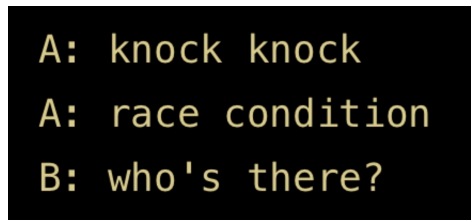
Hammad and Claire fighting for the last free slot

	Hammad	Claire
`t1`	check overlap: none!	
`t2`		check overlap: none!
`t3`	insert booking (ok)	
`t4`		insert booking (ok)
	Two bookings for one room and one slot.	

Each request ran correct code from start to finish. The bug is in the order their steps mixed (the interleaving).

Check-then-act

- When state is shared between threads, look in review for code that **checks** a condition, then **acts** on it in a separate step.
  - `if (isFree(slot)) book(slot);`
  - `if (!exists(file)) create(file);`
  - `if (balance >= amount) withdraw(amount);`
- The check's answer can be out of date by the time the act runs.
- A bug whose outcome depends on the timing of threads is called a **race condition** (or just a race).



ProgrammerHumor.io

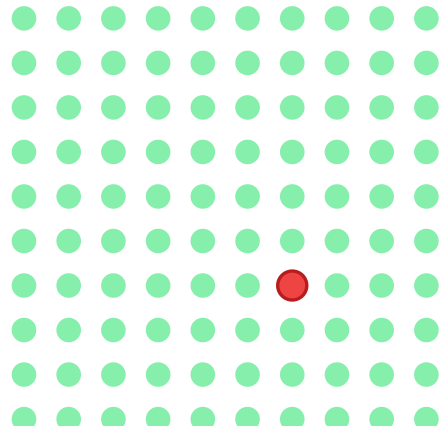
Lost updates

```
// each student's active-booking count, for the booking limit
int n = active.get(student);       // read
active.put(student, n + 1);        // write
```

- Read-modify-write is another common race.
  - Even `count++` is three steps (read, add one, write).
- Two bookings by one student at once can both read 2 and both write 3. One booking is never counted, and (usually) no error is reported.
- This is the lost update we saw last time, but inside one copy. It can let a student go past the booking limit.

Why didn't the tests catch it?

- The suite calls `book()` one request at a time, so it sees one interleaving.
- The race needs two requests inside a gap that may last microseconds.
  - Recall that a test needs to control its inputs (controllability). The order of threads is an input an ordinary test doesn't control.
- A test that starts two threads may hit the gap only once in many runs.
  - A test that fails only some of the time is called **flaky**. (A flaky test can point to a real bug in the code.)



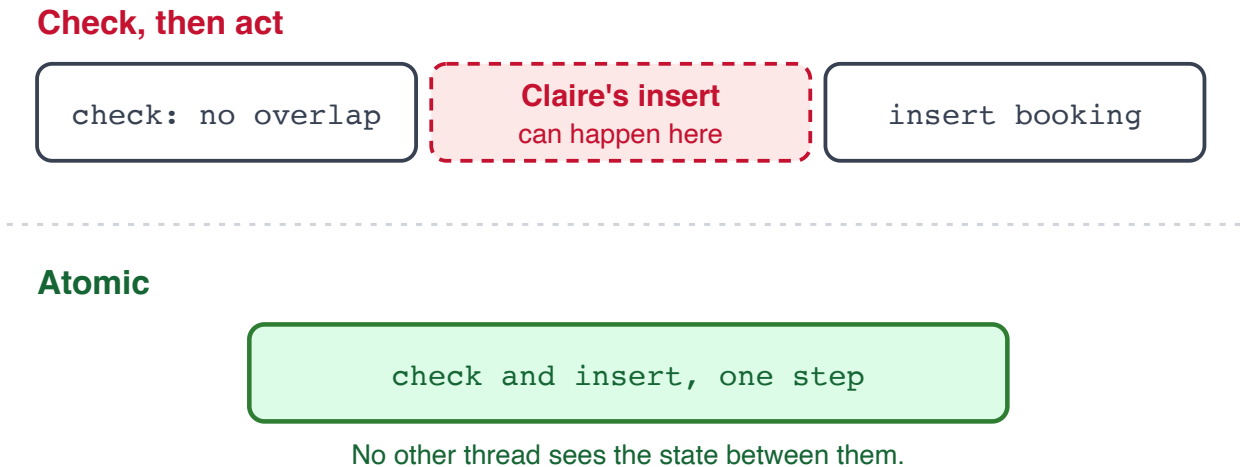
A two-thread test, 100 runs

Shared mutable state

- A race needs data that is both **shared** (two threads can reach it) and **mutable** (it can change).
- That leaves three ways out:
  - Synchronize every access to the data.
  - Don’t change the data.
  - Don’t share the data.

Remedies

Atomic operations



An operation is **atomic** when every other thread sees all of it or none of it.

One call or two?

```java
// the count from Lost updates, in Java's thread-safe map
Map<Student, Integer> active = new ConcurrentHashMap<>();

// Version A: a get, then a put (two calls)
active.put(student, active.getOrDefault(student, 0) + 1);

// Version B: merge adds 1 to the current count (one call)
active.merge(student, 1, Integer::sum);
```

- Many languages have thread-safe data structures built in (e.g., C#'s `ConcurrentDictionary`, Python's `queue.Queue`).
- A `ConcurrentHashMap` makes each single-key call on it atomic, but not a sequence of calls. Version A can still lose an update.
- Methods like `merge` and `compute` read and write in one atomic call.
- When a map is shared between threads, a `get` followed by a `put` on the same key is worth a second look in review.

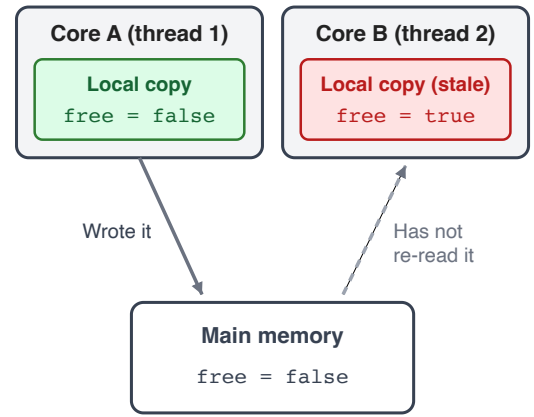
Locks

```
synchronized (bookingLock) {
  if (hasActiveOverlap(r, slot)) throw new SlotTaken();
  return store.insert(new Booking(u, r, slot));
}
```

- A lock lets one thread at a time into a region of code. The others wait until it's released.
- `synchronized` is Java's built-in lock.
  - On a method, `synchronized` locks the object the method is called on (`this`).
  - A `ReentrantLock` does the same job with explicit `lock()` and `unlock()` calls, and can give up after a timeout.
- Other languages have the same idea (e.g., Python's `threading.Lock` was in the agent plan we saw last time).

Visibility

- Without synchronization, a thread may not see another thread's write, or may see writes in a different order than they happened.
  - (CPU and compiler optimizations cause this, for example by keeping a value in a register.)
  - This holds in most languages where threads share memory (e.g., Java, C++, Go).
- Locks and atomic variables (like Java's `AtomicLong`) make a write visible to the next thread that takes the same lock or reads the variable.
- (As a reminder, a stale cache can give stale reads. This is the same problem inside one machine.)



Let's lock everything?

- Too much locking can hurt performance. Threads may spend their time waiting, or the program can end up running one thread at a time.
- With one lock for all bookings, each booking waits while another booking holds the lock, even one for a different room.
- One lock per room lets bookings for different rooms run in parallel, but it's more code to get right.
  - Do note that a rule that spans rooms (e.g., the booking limit) isn't protected by per-room locks.
- For any fix to a race, it is not a bad idea to ask what now runs one at a time that used to run in parallel.



Deadlock

```
Thread 1: lock(roomA) ... waiting for lock(roomB)
Thread 2: lock(roomB) ... waiting for lock(roomA)
```

- Each thread holds the lock the other needs, so neither can continue. The program doesn't crash, and usually no error is logged.
- For example, two threads swapping bookings between two rooms can take the room locks in opposite orders.



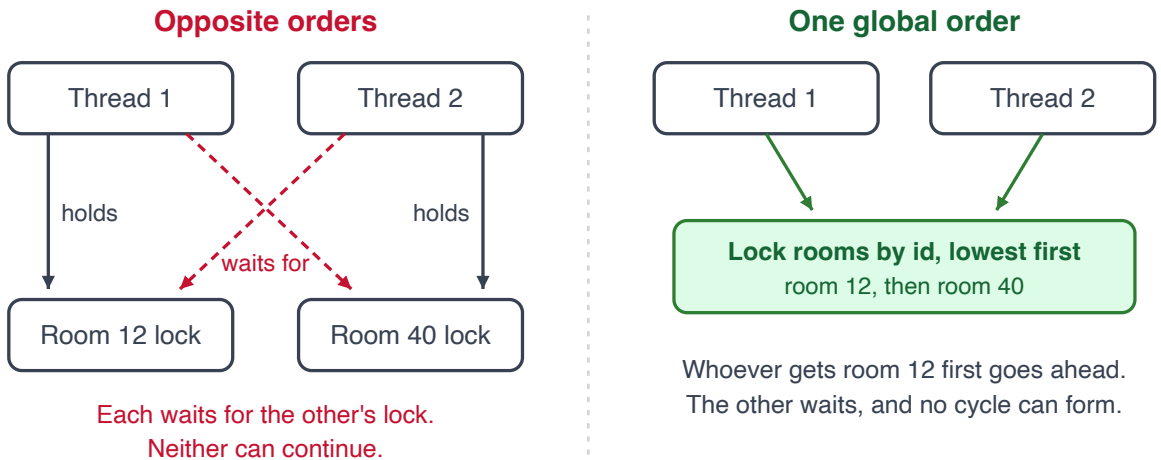
ProgrammerHumor.io

Conditions for deadlock

Deadlock needs all four of these at once:

- **Mutual exclusion:** Only one thread can hold a lock at a time.
- **Hold and wait:** A thread holds one lock while it waits for another.
- **No preemption:** A lock can't be taken away from the thread holding it.
- **Circular wait:** Each thread waits for a lock the next one holds, around a cycle.
- Breaking any one of them prevents deadlock.
  - For instance, taking locks in one fixed order breaks the cycle. And a timeout that releases your locks also works.

Avoiding deadlock



If one change needs two locks, it can also be worth asking whether the design should change instead.

Safety, liveness, and performance

- **Safety:** Nothing bad happens.
  - A race breaks it (two bookings for one slot).
- **Liveness:** Something good eventually happens.
  - A deadlock breaks it (the threads stop making progress).
  - A timeout on a blocking call (taking a lock, a network read) turns a hang into an error you can handle.
- **Performance:** The good thing happens quickly.
  - Too much locking breaks it.
- A fix for one can break another. A lock fixes a race, but it can slow things down. Two locks can deadlock.

Letting the database enforce it

```
ALTER TABLE bookings ADD UNIQUE (room, slot);
```

- This is the constraint we saw last week. The database refuses a second booking for the same room and slot.
- The check and the insert become one atomic step inside the database. Claire's insert after Hammad's fails, whatever the interleaving.
- Note that a unique key only refuses an exact repeat. Our bookings are time ranges. Two ranges can overlap without being equal (10:00 to 11:00 and 10:30 to 11:30).
  - Postgres has an **exclusion constraint** for that, which refuses two active bookings with overlapping slots in the same room.

A lock or the database?

- If the database can state the rule (a unique key, no overlapping slots), it's usually the better place for it. It holds for every copy of the service and every code path that writes, as long as one database primary takes the writes.
- Some rules aren't a constraint, like the booking limit (since it counts rows). One conditional update can still check and write in one step.
- A lock in our code fits when the shared state lives only in one copy (its memory or its files). For example:
  - In each copy's availability cache, a lock stops two threads from rebuilding the same entry at once.
  - In a report file, a lock stops two threads' lines from interleaving.

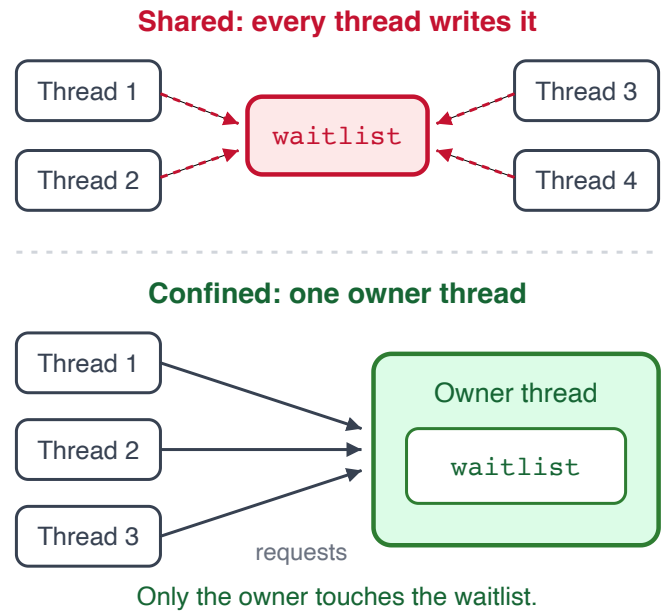
```
UPDATE students SET active = active + 1
 WHERE id = $1 AND active < 3;     -- 0 rows updated: over the limit
```

Immutability

- Data that never changes after it's built can be shared freely, without locks or stale copies.
- For example, a Java `record TimeRange(Instant start, Instant end)` can't change after it's built (`Instant` can't either). Any thread can hold one safely.
  - Notably, a record that holds a `List` can still change through that list, unless its constructor copies it (`List.copyOf`). Everything an immutable object holds should be immutable too.
- Append-only data works the same way. Once an event is written to the log, it's only ever read.

Confinement

- If state has to change, it can have one owner. One thread touches it, and other threads send that thread requests.
- With no sharing, there are no interleavings on that data, so sequential reasoning works for it again.
- As a reminder, each piece of data should generally have only one owner. Confinement applies that rule to threads.



Documenting thread safety

- Whether a class is safe to share between threads is part of its spec, so it should be written down.
  - For example, a class comment like "Safe for concurrent reads. Writes need a lock."
- Without it, the next caller (a person or an agent) may have to guess.
- An annotation can make the claim checkable. With `@GuardedBy("lock")` on a field, Error Prone (a linter for Java, run by the compiler) flags any access made without that lock held.

Let's practice: the booking limit

The booking system keeps each student's active-booking count in memory, and `book()` checks it against the limit before inserting. Two bookings by one student at once can both pass the check. Three fixes are proposed:

1. Put the count check, the insert, and the count update in one `synchronized` block.
2. Keep the count in Postgres, with one conditional `UPDATE ... WHERE active < 3`.
3. Change the count map to a `ConcurrentHashMap`, and keep the code as it is.

With a neighbor: which fix would you accept today, with one copy? Would your answer change at three copies?



Canvas → Quizzes → Lecture 17 In-Class Quiz

Asynchrony

Asynchrony and concurrency

- **Asynchrony** means starting slow work without blocking on it, and saying what should happen when it finishes.
- For example, the confirmation email and the search-index update shouldn't hold up the booking response.
- Asynchrony isn't the same as concurrency. It leads to concurrency when other work runs while we wait, which is usually why we use it.
  - When it does, the hazards from earlier apply.

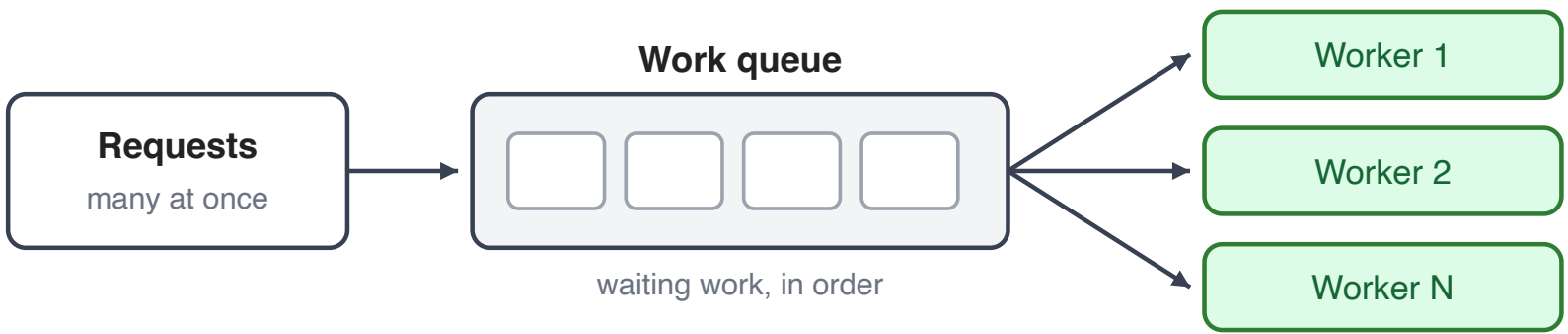
Awaiting slow work

```
const booking = await bookingService.book(user, room, slot);
notifier.send(user, confirmationFor(booking))   // not awaited
  .catch(logFailure);
```

- A **promise** is a value that arrives later. `await` pauses this function until it arrives, and other work runs in the meantime.
- An **async function** is a function that can pause partway (at each `await`) and finish later. Calling one gives back a promise for its result right away.
- Python, C#, and Rust use `async` / `await` too, with small differences in syntax and semantics. The idea is the same.
- An **event loop** runs many paused tasks on one thread, and resumes each one when its result arrives. (JavaScript works this way.)

Thread pools and work queues

- Starting a new handler for every request can overload a server when thousands arrive at once.
- A thread pool bounds this with a fixed number of workers. The rest of the work waits in a queue.
- A queue between two components also lets them run at different speeds.
  - (We'll see queues again next week.)



Events and concurrency

- Recall that Observer lets a publisher notify its subscribers without knowing who they are.
- In a running system, the subscribers to `BookingCancelled` may run at the same time as each other, and as the next booking.
  - For example, promoting the next student from the waitlist can race with a new booking for the freed slot.
- Loose coupling doesn't remove interleavings. The hazards from earlier apply to subscribers too.

Async hazards

```
async function cancelBooking(id: BookingId) {
  api.cancel(id);   // not awaited: returns before the cancel finishes,
}                   // and a failed cancel never reaches the caller
```

- **Lost errors:** A promise that isn't awaited can fail without the caller seeing the error.
  - `await api.cancel(id)` waits for the cancel to finish (other requests still run meanwhile), so a failure reaches the caller.
- **Ordering:** Results arrive in the order the work finishes, which may differ from the order it started.
- **Half-done work:** The booking was written, the notification wasn't, and the process crashed in between. (We'll see what to do about this next week.)



ProgrammerHumor.io

Races in async code

```
async function book(user: UserId, room: RoomId, slot: Slot) {
  const free = await bookings.isFree(room, slot);   // check
  if (!free) throw new SlotTaken();
  await bookings.insert(user, room, slot);          // act
}
```

- Async tasks run on one thread, but each `await` lets other requests run before this one continues.
- Two requests can both see `free` as true before either inserts. This is the same check-then-act race as with threads.
  - The same remedies apply. For example, the database constraint still rejects the second booking.

Generated concurrent code

An agent's thread-safety claim

The task: "Make `WaitlistService` safe to call from many request threads".

```
/** Thread-safe: join is synchronized. */
class WaitlistService {
  private final WaitlistStore store;
  WaitlistService(WaitlistStore store) { this.store = store; }
  synchronized void join(Student s, Room r, TimeRange slot) {
    if (store.countWaiting(r, slot) >= MAX_WAITING) throw new WaitlistFull();
    store.add(new Entry(s, r, slot));
  }
}

// WaitlistController, which the agent didn't change
void handleJoin(JoinRequest req) {
  new WaitlistService(store).join(req.student(), req.room(), req.slot());
}
```

The agent's report: "Made `join` synchronized, so two requests can't interleave inside it. Added a test where 50 threads share one `WaitlistService` and join at once. It passes."

Why the claim doesn’t hold

- A `synchronized` method locks the object it’s called on. The controller builds a new `WaitlistService` per request, so two requests take two different locks. Both can count, then both add, and the waitlist goes past `MAX_WAITING`.
- A lock guards only the threads that take the same lock.
- The test shares one service, so its threads do take the same lock. It passes because its setup differs from production.
- The agent’s code was right for the setup it assumed. The setup was in a file it didn’t read.

Remember Jenkins?

From `XmlFile.java`, in the subsystem you refactored for A4. Why is one of these fields a `ThreadLocal` and the other a synchronized map? (A `ThreadLocal` gives each thread its own copy of a value.)

```
private static final Map<Object, Void> beingWritten =
        Collections.synchronizedMap(new IdentityHashMap<>());
private static final ThreadLocal<File> writing = new ThreadLocal<>();
...
beingWritten.put(o, null);
writing.set(file);
try {
    xs.toXML(o, w);
} finally {
    beingWritten.remove(o);
    writing.set(null);
}
```

Jenkins 2.568.1, github.com/jenkinsci

Two kinds of thread safety in one file

- `writing` (a `ThreadLocal`) holds the file that **this thread** is saving. Each thread has its own copy, so the field is confined and doesn't need a lock.
- `beingWritten` is one map that every thread shares, so every access to it goes through a synchronized wrapper.
- A comment elsewhere in the file says what the fields are for, but nothing in the file says why one is per-thread and the other shared.
  - A one-line comment at each declaration would tell the next person to change this code (or their agent).
- A4's tests run one save at a time. A refactor that made `writing` a plain static field, or the map a plain `HashMap`, would very likely still pass them.

Checking concurrent code

For most code, we check generated work with tests and review where the risk is. For concurrent code, a passing test has usually seen one interleaving, or a few.

- **Stress tests:** Run many threads many times, then check the invariant. They can't show a race is absent, but they make one more likely to show up.
- **An inventory:** Have the agent list each piece of shared mutable state and what guards it, then check the list. If an entry has no guard, it's worth raising in review, even when the tests pass.
- **Forced interleavings:** Pause one thread at a chosen point, run the other, then check the invariant.
- As a reminder, kinds of checks differ in strength (a proof, then tests, then reading). A database constraint is closer to a proof than a stress test is.

Making a race fail every run

- Recall that we made time testable by passing a `Clock` into `BookingService`.
- The order of threads is an input too. A test can set it by pausing one thread at a chosen point.

	Thread A	Thread B
`t1`	count: one below the limit (then pauses)	
`t2`		count: one below the limit, add (list full)
`t3`	resumes, add	
	One entry past `MAX_WAITING`, on every run.	

A test that forces the interleaving

`PausingStore` (Lab 10 gives you one) pauses the chosen call until `release()`. `tryJoin` calls `join` and ignores `WaitlistFull`.

```
@Test
void twoJoinsCannotOverfillTheWaitlist() throws Exception {
  fillToOneBelowTheLimit(store, room, slot);
  var paused = new PausingStore(store, "countWaiting", 1);  // pauses the 1st countWaiting
  // one service per request, as the controller does
  Thread a = new Thread(() -> tryJoin(new WaitlistService(paused), hammad));
  a.start();
  paused.awaitPaused();                          // A has counted, but not added
  tryJoin(new WaitlistService(paused), claire);  // B counts and adds: list full
  paused.release();                              // A adds, using its old count
  a.join(2_000);
  assertTrue(store.countWaiting(room, slot) <= MAX_WAITING);
}
```

This fails on every run while `join` has the race. A pass covers this one interleaving, so other interleavings need their own test or a stress test.

Races in A5

- The A5 checkpoint (Friday) asks for checks that fail on what replicas break. Some of what replicas break can also break inside one replica, between threads.
- If your service checks something and then acts on it (e.g., "is this slot free?", then insert), add a harness check that sends many of those requests at once and then checks the invariant.
- When the state lives in the database, the list of shared state inside one process is usually short (a cache, a counter, etc.). For *most* designs, you should hopefully not be stuck reviewing a long list.
- A5 doesn't expect every interleaving. A race that shows up within a few hundred attempts is the level it means.
- As a reminder, a lock in one process guards only that copy.

Summary

- Concurrency can come with users, background work, and networks, whether or not our code starts a thread.
- Check-then-act and read-modify-write on shared mutable state are two common races. A test usually sees one interleaving.
- Remedies (locks, database constraints and conditional updates, immutability, confinement) make the check and the act atomic, or avoid shared mutable state. Too much locking can slow things down, and two locks can deadlock.
- Async/await is a common way to write concurrent code. Await (or handle) every promise, and treat each `await` as a point where another request can run.
- A lock guards only the threads that take the same lock. For concurrent code, have the agent list the shared state and its guards, and force the interleaving you're worried about.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due		A4 due	M2 · A5 chkpt		A5 due		A6 due

- **Wednesday: Midterm 2.** Today's content is not in scope.
- **Friday, Lab 10:** Take a race from a bug report to a test that fails on every run, judge a teammate's fix, then choose your own.
- **Friday, the A5 checkpoint:** Checks that fail on what replicas break, a `SCALE-AUDIT.md`, your throughput at one replica, and a first draft of `NOTIFICATIONS.md`.
- **Next Monday:** Coordinating several services, when there's no shared memory or shared transaction.